arXiv:2602.21788v1 [cs.DC] 25 Feb 2026

DHP: Efficient Scaling of MLLM Training with Dynamic Hybrid Parallelism

Yifan Niu [*1] **Han Xiao** [*1] **Dongyi Liu** [1] **Wei Zhou** [2] **Jia Li** [1 3]

Abstract

Scaling long-context capabilities is crucial for Multimodal Large Language Models (MLLMs). However, real-world multimodal datasets are extremely heterogeneous. Existing training frameworks predominantly rely on static parallelism strategies, which suffer from severe load imbalance, redundant communication, and suboptimal hardware utilization under data heterogeneity. In this work, we propose **D**ynamic **H**ybrid **P**arallelism (DHP), an efficient parallelism strategy that adaptively reconfigures communication groups and parallelism degrees during MLLM training. We generalize the non-power-of-two parallelism degrees and develop a polynomial-time algorithm to generate near-optimal parallelism strategies with only millisecond-level overhead per training batch. DHP is able to maintain high hardware efficiency even under extreme data variability. Experimental results demonstrate that DHP significantly outperforms Megatron-LM and DeepSpeed, achieving up to 1.36 $\times$ speedup in training throughput while maintaining near-linear scaling efficiency across large-scale NPU clusters.

1. Introduction

In recent years, Multimodal Large Language Models (MLLMs) have demonstrated remarkable success in achieving unified understanding and generation across diverse modalities (Liu et al., 2023; Awadalla et al., 2023; Cheng et al., 2024; Xu et al., 2025; Zhang et al., 2025; Li et al., 2025). The impressive performance of MLLMs is primarily attributed to the expansion of model parameters, the integration of massive-scale interleaved vision-language datasets, and the adoption of higher visual resolutions, all of which align with the multimodal scaling laws. This has driven the development of state-of-the-art models with enhanced multimodal capabilities. For instance, industry-leading models such as OpenAI's GPT-5 (Singh et al., 2025) and Google's Gemini 3 Pro (Anil et al., 2025) exhibit unprecedented native multimodal reasoning and agentic capabilities. Simultaneously, open-source models like Qwen3-VL (Bai et al., 2025) and InternVL-3.5 (Wang et al., 2025a) have pushed the boundaries of high-resolution visual processing and extensive multimodal context windows.

A fundamental challenge in scaling MLLMs to high-resolution images or long videos is the quadratic complexity $O(L^2)$ of self-attention regarding the number of tokens (Yang et al., 2024; Shao et al., 2025a;b). Furthermore, compared to casual attention, full attention in the vision encoder requires twice the computational effort. Thus, the computational workload is also dictated by the shape and sparsity of the full attention mask. Existing acceleration strategies for training are generally divided into two categories: static and dynamic parallelism. Static parallelism (e.g., Megatron-LM (Narayanan et al., 2021), DeepSpeed (Rasley et al., 2020)) typically employs 4D parallelism: Data Parallelism (DP) (Dean et al., 2012; Li et al., 2020), Tensor Parallelism (TP) (Shoeybi et al., 2020) , Pipeline Parallelism (PP) (Huang et al., 2019; Narayanan et al., 2019; 2020), Sequence Parallelism (SP) (Brandon et al., 2023; Li et al., 2023b; Jacobs et al., 2024) or Context Parallelism (CP) (Brandon et al., 2023; Li et al., 2023a) to distribute the training on massive clusters. However, static methods often suffer from workload imbalances and redundant communication with heterogeneous sequences. To address the issue, recent research has introduced dynamic parallelism. ByteScale (Ge et al., 2025) proposes a heuristic-based sequence scheduling strategy. FlexSP (Wang et al., 2025b) adjusts the data partition by solving a time-consuming programming problem and restricts the communication group size to powers of two.

In this work, we follow the trajectory of dynamic parallelism and rethink the essential properties that define an optimal scheduling strategy. A robust dynamic parallelism should satisfy three core requirements: (1) **Workload Balance**. The computational load across different communication groups must be meticulously balanced to minimize synchronization stalls and maximize hardware efficiency. (2) **Elastic Parallelism Degree**. The strategy should support

[*]Equal contribution [1]The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China [2]Huawei Technologies Co., Ltd. [3]The Hong Kong University of Science and Technology, Hong Kong, China. Correspondence to: Jia Li <jialee@ust.hk>.

Preprint.

flexible, non-power-of-two parallelism degrees to prevent short sequences from incurring redundant communication overhead, thereby adapting to highly skewed data distributions. (3) **Minimal Scheduling Overhead**. The latency of generating an assignment strategy must be sufficiently low to be fully overlapped with the computation of sequences of any length, ensuring that the scheduling process itself does not become a new bottleneck.

Motivated by these principles, we propose Dynamic Hybrid Parallelism (DHP), an efficient and adaptive framework designed to optimize MLLM training on heterogeneous datasets. DHP dynamically reconfigures communication groups and parallelism degrees for each micro-batch to handle the skewed distributions of video and image data. DHP leverages the flexibility of Ring-style Context Parallelism (CP), allowing for arbitrary integer parallelism degrees that better fit the varying sequence lengths. To solve the NP-hard scheduling problem without introducing training latency, we develop a two-stage algorithm to obtain approximated solutions: Memory-aware Sequence Packing to group sequences, followed by a 2D-Dynamic Programming approach to find the optimal resource allocation. We implement DHP as a non-intrusive, asynchronous module atop Megatron-LM, ensuring that scheduling overhead is fully hidden behind NPU computation. Experimental results demonstrate that DHP significantly outperforms Megatron-LM and DeepSpeed, achieving up to 1.36 $\times$ speedup in training throughput while maintaining near-linear scaling efficiency across large-scale NPU clusters.

2. Related Work

Parallelism for Large-Scale Training. Distributed training on massive clusters is indispensable for large-scale model training. Existing parallelism strategies can be broadly categorized into static and dynamic parallelism. Static parallelism (e.g., Megatron-LM (Narayanan et al., 2021), DeepSpeed (Rasley et al., 2020)) typically employs 4D parallelism: Data Parallelism (DP) (Dean et al., 2012; Li et al., 2020) replicates model weights across workers and partitions data; Tensor Parallelism (TP) (Shoeybi et al., 2020) partitions weight matrices within layers; Pipeline Parallelism (PP) (Huang et al., 2019; Narayanan et al., 2019; 2020) distributes model layers across different stages and processes micro-batches in a pipeline; Sequence Parallelism (SP) (Brandon et al., 2023; Li et al., 2023b; Jacobs et al., 2024) or Context Parallelism (CP) (Brandon et al., 2023; Li et al., 2023a) partitions along the sequence dimension to support long-context training. However, static methods often suffer from workload imbalances and exhibit poor adaptability to heterogeneous sequence lengths. To address these issues, recent research has introduced dynamic parallelism. ByteScale (Ge et al., 2025) employs data-aware sharding and dynamic communication to eliminate redundant communication for short sequences. FlexSP (Wang et al., 2025b) adjusts partitioning by solving the time-consuming Mixed-Integer Linear Programming problem.

MLLM Training Acceleration. Recent studies have proposed several strategies to accelerate MLLM training, which can be categorized into two classes: algorithm-based and system-based approaches. Algorithm-based optimization primarily focuses on model architecture and training strategies. In terms of model architecture, Chain-of-Sight (Huang et al., 2024) incorporates a visual resampler to replace global cross-attention with multiple local window attentions. This reduces training computational costs while maintaining model performance. Regarding training strategies, Pyramid-Drop (Xing et al., 2025) achieves a reduction in overall computation by progressively pruning redundant vision tokens through staged token pruning. On the other hand, system-based optimizations mainly address the bubble introduced by the complex architecture of MLLMs. Optimus (Feng et al., 2025) schedules the Vision Encoder to compute within the bubbles to accelerate training. However, current research has not addressed the significant load imbalance and redundant computation that exist in current MLLM training on complex heterogeneous multimodal data.

3. Preliminaries

3.1. Multimodal Large Language Models

A typical Multimodal Large Language Model (MLLM) can be abstracted into three modules: a pre-trained modality encoder, a cross-modal connector, and a large-scale language model (Radford et al., 2021; 2023; Bai et al., 2025). Formally, given an input image I, the visual encoder $\mathcal{E}_v$ extracts high-level features $Z_v = \mathcal{E}_v(I)$. The connector ϕ then projects these features into the language embedding space, yielding visual tokens $H_v = \phi(Z_v)$. These are concatenated with text query embeddings H_q to form the input sequence $H_{in} = [H_v; H_q] \in \mathbb{R}^{L \times D}$, where L is the token count and D is the hidden dimension. The language model processes H_{in} primarily through the Self-Attention (Vaswani et al., 2017). The attention operation is defined as:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{1}$$

where Q, K, V are query, key, value projections. Computing the affinity matrix QK^T incurs a computational complexity of $\mathcal{O}(L^2)$ regarding the number of tokens. The language model inserts visual tokens into text tokens, enabling it to perform multimodal reasoning according to both visual and linguistic contexts. Finally, the model generates the response by modeling the conditional probability in an auto-regressive manner.

3.2. Distributed Training Paradigms

Data Parallelism. Data Parallelism (DP) (Li et al., 2020) partitions data along the batch dimension. Each device maintains a full replica of model parameters and synchronizes gradients via All-Reduce operations. To mitigate the redundancy of model states, Sharded Data Parallelism (SDP) (e.g., ZeRO (Rajbhandari et al., 2020), FSDP (Zhao et al., 2023)) partitions model states across devices, trading increased communication for significantly reduced per-device memory consumption.

Model Parallelism. Model Parallelism (MP) decomposes model parameters across devices, categorized into Tensor Parallelism (TP) and Pipeline Parallelism (PP). TP (Shoeybi et al., 2020) partitions individual operators, requiring frequent synchronization of activations. PP (Huang et al., 2019; Narayanan et al., 2019) partitions the model by layers and only transfers boundary activations between stages, making it suitable for cross-node scaling.

Sequence-Dimension Partitioning (CP & SP). Both Sequence Parallelism (SP) and Context Parallelism (CP) address memory bottlenecks by distributing data along the sequence dimension. Sequence Parallelism (SP), such as DeepSpeed-Ulysses (Jacobs et al., 2024), uses All-to-All primitives to redistribute data so that each device computes attention for a subset of heads over the full sequence. Context Parallelism (CP) (e.g., Ring-Attention (Brandon et al., 2023; Li et al., 2023b; 2024)) decomposes the attention computation itself. CP often employs ring-based communication to exchange Key/Value blocks, allowing the overlap of communication with computation.

4. Method

4.1. Problem Formulation

Multimodal datasets exhibit a skewed, long-tail distribution in video duration—most videos are under 8 seconds, while few exceed 64 seconds—with significant variability across datasets like MSRVTT (Xu et al., 2016), InternVid (Wang et al., 2024), and OpenVid (Nan et al., 2025), as shown in Figure 1. However, static communication grids partition devices uniformly, leading to computation and load imbalances when handling such heterogeneous data.

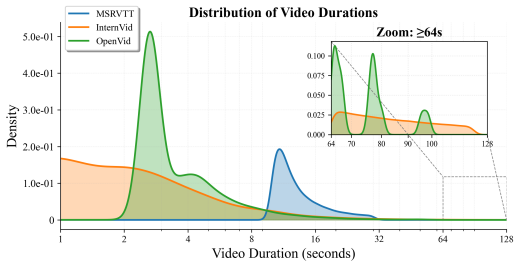

Figure 1. Data distribution in MSRVTT, InternVid, and OpenVid.

In this work, we focus on the dynamic construction of CP groups, as the ring attention overlaps the communication cost well for long sequences. By dynamically assigning each sequence s_k in a micro-batch $\mathcal{B}$ to a specific CP group C_p with an appropriate parallelism degree d_p, our formulation implicitly incorporates DP. We exclude Model Parallelism (TP & PP) from our dynamic optimization space, as the overhead of reconfiguring TP/PP groups (e.g., weight reshuffling and communication ring rebuilding) during runtime is unacceptable. Consequently, we treat TP and PP as predefined static configurations. In our discussion, a single "rank" r_n represents a complete model replica (i.e., TP $\times$ PP physical NPUs), and N denotes the total number of such replicas available in the cluster.

Formally, let $\mathcal{R}$ be the set of N available computing units in the cluster. We aim to dynamically partition $\mathcal{R}$ into disjoint CP groups $\mathcal{C} = \{C_p\}^P$ with degree d_p, and $\bigcup C_p = \mathcal{R}$. Given a micro-batch of K sequences $\{s_k\}$ with heterogeneous lengths and a per-rank memory budget E, we aim to determine: (1) the total number of CP groups, (2) the parallelism degree of each group, and (3) the assignment of each sequence to a specific CP group. Here, we extend the optimization objective in FlexSP to a more flexible and generalized form by incorporating Ring-style CP. In Ulysses-style SP (Jacobs et al., 2024), the parallel degree must divide the number of attention heads. In practice, this often restricts the SP degree to powers of two, limiting the search space for optimal configurations. In contrast, we adopt Ring-style CP (Brandon et al., 2023) that allows the CP degree d_p to be any positive integer. This relaxation enables finer-grained resource allocation and better adaptation to heterogeneous sequence lengths. We define the sequence assignment matrix $\boldsymbol{A} \in \{0,1\}^{K\times P}$, where $A_{k,p} = 1$ represents that $\mathcal{S}_k$ is assigned to $\mathcal{C}_p$. We define the generalized optimization problem for CP group and sequence assignment as follows:

$$\arg\min_{\boldsymbol{A},\mathcal{C}} \max \mathcal{T}(\mathcal{C}_p), \tag{2}$$

$$\text{s.t.} \quad \mathcal{M}(\mathcal{C}_p) \le E \cdot d_p,\ \forall p \in [1,P] \tag{3}$$

$$\sum\nolimits_k A_{k,p} \le K,\ \forall p \in [1,P] \tag{4}$$

$$\sum\nolimits_p A_{k,p} = 1,\ \forall k \in [1,K] \tag{5}$$

$$\sum\nolimits_p d_p \le N \tag{6}$$

where $\mathcal{T}(\cdot)$ and $\mathcal{M}(\cdot)$ denotes the execution time and the memory consumption in CP group $\mathcal{C}_p$. The optimization objective is to minimize the makespan, defined as the maximum execution time across all CP groups. The constraints are defined as follows: Cond. (3) enforces the hardware memory limit on each rank; Cond. (4) and (5) guarantee the exclusive assignment of each sequence to exactly one active CP group; finally, Cond. (6) ensures that the aggregate parallelism degree across all selected groups does not

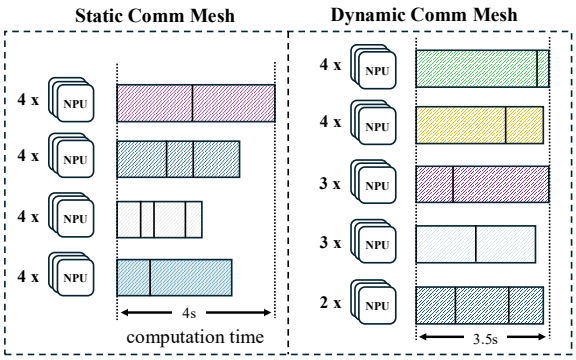

Figure 2. Static Mesh v.s. Dynamic Mesh.

exceed the total rank budget N. Note that directly solving this optimization problem is an NP-hard problem.

Figure 2 illustrates the core advantage of dynamic mesh over static meshes. In static mesh, fixed parallelism degrees lead to significant idle gaps and synchronization stalls due to the heterogeneity of sequences. Conversely, dynamic mesh adaptively reconfigures communication groups. By avoiding over-packing of short sequences and balancing the computational workload, it minimizes hardware idle time and reduces the total computation time.

4.2. Cost Estimation

Memory Estimation. The memory consumption consists of two primary components: model states and activations. For model states, we use ZeRO-3 redundancy elimination (Rajbhandari et al., 2020), and the model state memory M_{ms} remains constant per rank. For activations, CP partitions each sequence s_k evenly into d_p chunks along the sequence dimension. Thus, for a CP group $\mathcal{C}_p$, the memory consumption is estimated as:

$$\mathcal{M}(\mathcal{C}_p) = \sum_k A_{k,p}|s_k| \cdot \mathrm{M}_{token} + \mathrm{M}_{ms}, \tag{7}$$

where M_{token} represents the activation memory per token.

Time Estimation. (1) **Computation Cost.** To adapt to MLLM workloads, we refine the computation cost estimation by considering the full attention. While tokens in LLM follow a causal attention, visual tokens in vision encoder often employ full attention. We define a mask efficiency factor η_k to represent the additional computation cost introduced by full attention mask. Thus the computation cost of group $\mathcal{C}_p$ can be represented as:

$$\mathcal{T}_{cp}(\mathcal{C}_p) = \sum_k A_{k,p}(\alpha_1(1+\eta_k)|s_k|^2 + \alpha_2|s_k|) + \beta_1, \tag{8}$$

where $\alpha_1, \alpha_2, \beta_1$ are profiled coefficients, and η_k can be determined by the shape of attention mask.

(2) **Communication Cost.** Ring Attention (Brandon et al., 2023) utilizes P2P communication in a circular topology to exchange Key-Value (KV) blocks. The total communication cost is proportional to the sequence length s_k. The total communication cost is estimated as:

$$\mathcal{T}_{cm}(\mathcal{C}_p) = \frac{1}{v_p}\sum_k A_{k,p}\alpha_3|s_k| + \beta_2, \tag{9}$$

where v_p is the P2P bandwidth within the CP group.

(3) **Total Time Estimation.** Since the ring attention allows the communication cost and computation cost of attention to overlap, the communication time of attention needs to be excluded here. For simplicity, we do not detail the formula of the attention computation time $\mathcal{T}_{cpa}$ and communication time $\mathcal{T}_{cma}$, which has the similar form as Formula (8) and (9). Then, the overall execution time is:

$$\mathcal{T}(\mathcal{C}_p) = \mathcal{T}_{cp} + \mathcal{T}_{cm} - \min(\mathcal{T}_{cpa}, \mathcal{T}_{cma}). \tag{10}$$

Algorithm 1: 2D-Dynamic Programming

Input: Groups $\mathcal{G}$, Min-degrees $\{d_{\min,k'}\}$, Total ranks N
Output: CP degrees $\{d_p\}_{p=1}^{P}$

Initialization;
$K' \leftarrow |\mathcal{G}|, DP[K'+1][N+1], Path[K'+1][N+1]$;
$DP[0][0] \leftarrow 0$;
for $k = 1$ **to** K' **do**
  $R_{remain} \leftarrow \sum_{z=k+1}^{K'} d_{\min,z}$;
  for $j = (\sum_{i=1}^{k} d_{\min,i})$ **to** $(N - R_{remain})$ **do**
    for $d = d_{\min,k}$ **to** $(j - \sum_{i=1}^{k-1} d_{\min,i})$ **do**
      $cost \leftarrow \max(DP[k-1][j-d], \mathcal{T}(\mathcal{C}_p, d))$;
      if $cost < DP[k][j]$ **then**
        $DP[k][j] \leftarrow cost$;
        $Path[k][j] \leftarrow d$;
$p \leftarrow K' + 1$;
$q \leftarrow N + 1$;
while $p, q > 0$ **do**
  $d_p \leftarrow Path[p][q]$;
  $p \leftarrow p - 1,\ q \leftarrow q - d_p$;
return $\{d_1, \ldots, d_P\}$;

4.3. Polynomial-Time Problem Solving

To efficiently solve the NP-hard problem defined in Eq. (2), we propose a two-stage approximation algorithm: Memory-aware Sequence Packing followed by Dynamic Programming Resource Allocation. This significantly reduces the search space to find an approximate optimal solution while ensuring hardware constraints are strictly met.

Stage 1: Atomic Sequence Grouping via Best-Fit Decreasing (BFD). To mitigate memory fragmentation and reduce the number of decision variables, we first group heterogeneous sequences into *atomic sequence groups*. We

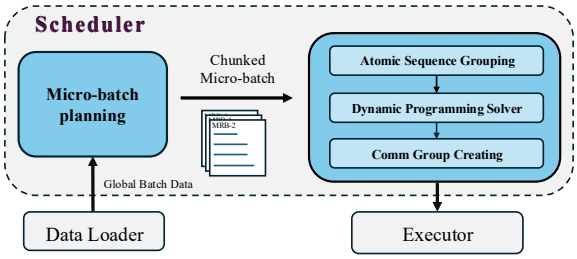

Figure 3. Overall workflow of DHP.

sort all sequences in $\mathcal{B}$ in descending order of their memory requirements. For each long sequence, we determine its minimum required CP degree $d_{\min,k'} = \lceil \mathcal{M}(s_k)/E \rceil$, effectively initializing a "bin" with capacity $d_{\min,k'} \cdot E$. We then employ a BFD strategy to greedily pack shorter sequences into the remaining memory headroom of these established bins. This transforms the original set of K sequences into K' atomic groups $\{\mathcal{G}_1, \ldots, \mathcal{G}_{K'}\}$, where $K' \leq K$. Each group $\mathcal{G}_k$ is treated as a single scheduling unit that requires at least $d_{\min,k'}$ ranks to satisfy memory constraint. Note that this avoids communication redundancy caused by packing massive short sequences.

Stage 2: Optimal Resource Assignment via 2D-Dynamic Programming. Given the atomic groups, we propose a 2D dynamic programming approach to determine the optimal CP degree d_p and assignment $A_{k,p}$ for each group. Let $DP[i][j]$ denote the minimum achievable *makespan* (i.e., the maximum execution time among all groups) for the first i atomic groups utilizing a total of j ranks. The state transition is defined as:

$$DP[i][j] = \min_{d \in [d_{\min,i}, j-d']} \max(DP[i-1][j-d], \mathcal{T}(\mathcal{G}_i, d)) \tag{11}$$

where $d' = \sum_{m=1}^{i-1} d_{\min,m}$ ensures that sufficient ranks are reserved for the preceding $i-1$ groups. The function $\mathcal{T}(\mathcal{G}_i, d)$ represents the estimated execution time of group $\mathcal{G}_i$ under CP degree d. The algorithm is shown in Alg. 1.

After populating the DP table, we **backtrack** from $DP[K'][N]$ to retrieve the optimal assignment matrix $\boldsymbol{A}$ and the corresponding CP degrees $\{d_p\}_{p=1}^{P}$. The total time complexity of this stage is $O(K'N^2)$ with only millisecond-level overhead, which can be overlapped with any sequence computation time.

5. Overall Workflow and Implementation

Workflow. We introduce the overall workflow of the DHP scheduler, as illustrated in Figure 3. (1) Given a training Global Batch, we first use **micro-batch planner** to chunk it into multiple micro-batches. (2) The scheduler first performs **memory-aware sequence packing** on micro-batch $\mathcal{B}$ to transform K individual sequences into K' atomic groups $\{\mathcal{G}_i\}$. (3) Subsequently, the scheduler invokes the **2D dynamic programming allocator** to determine the optimal assignment matrix $\boldsymbol{A}$ and CP degrees $\{d_p\}$ for each group. (4) After the DP solver is complete, the **executor** reallocates the hybrid parallel groups and dispatches data to each group.

Implementation Details. Our implementation is built upon **Megatron-LM** as the underlying training infrastructure. A key design principle is non-intrusiveness: we implement all DHP-specific components as add-on modules without modifying the megatron-core codebase. This ensures compatibility and easier maintainability. Our system primarily consists of two new core classes: **Profiler** and **Scheduler**, which together realize the dynamic parallelism. For communication components, our implementation is based on the HCCL communication backend provided within the PyTorch distributed package. Below, we detail several key aspects of the DHP implementation:

(1) **Dynamic Group Management and Pooling.** To achieve dynamic reconfiguration of context parallel communication groups, the DHP Scheduler recreate the communication groups and correspondingly update the parallel state (MPU). Creating new HCCL communication groups on the fly for each batch would significantly increase buffer overhead in the communication backend, further leading to errors. To avoid this, the DHP Scheduler maintains a communication group pool created at the beginning of training. This pool caches previously established communication groups. When constructing a new communication group, the Scheduler first checks the pool for a matching group; a new group is instantiated only if no suitable cached group is found. In practice, the total number of unique groups required is limited, and the creation overhead becomes negligible over long training runs.

(2) **Decoupling Scheduling and Training.** The workflow for each training batch involves two phases: (1) the scheduling phase, which determines the optimal parallel strategy and sequence assignment, and (2) the execution phase, which performs the actual forward/backward propagation using the assigned strategy. In DHP's implementation, these two phases are executed asynchronously, with the scheduling logic designed to run on the CPU while the NPU continues with model computations. Specifically, while the NPU processes the current batch, the CPU concurrently analyzes the token lengths of the next batch, predicts costs via the Profiler, solves for the optimal plan, and prepares the necessary communication groups. This producer-consumer pattern effectively hides scheduling latency.

(3) **Profiler Integration and Cost Modeling.** The Profiler class is central to DHP's adaptability. Before the whole training process, the core profile function first constructs data of different lengths for forward/backward passes and collects execution times for various sequence lengths under different SP degree. The Profiler will establish the func-

tional relationship between runtime and factors (sequence length and SP degree), storing the corresponding parameters within the Profiler class. When the profiler is queried, it predicts the cost based on the params stored. This facilitates fast, low-overhead cost estimation for the scheduler subsequently, which enables the dynamic programming solver to quickly evaluate the time cost of different sequence groupings.

(4) **Integration with Megatron Parallel Utilities.** DHP leverages and extends Megatron's parallel state manager (MPU). We added interfaces in the Scheduler class to dynamically update context parallel communication groups and reallocate the required data for each group, ensuring each NPU rank receives the correct data shard according to the latest dynamic plan. This implementation also ensures that all other forms of parallelism (e.g., TP, PP) continue to operate as defined in the original static grid, while DHP dynamically orchestrates CP and DP communication groups on top of it. This approach allows DHP to run seamlessly on top of existing Megatron training scripts.

6. Experiments

6.1. Experimental Setup

Hardware Environment. All experiments are conducted on a cluster of 8 nodes. Each node is equipped with 8 Ascend 910B NPUs (64GB memory each), which are interconnected via HCCS within the node. Inter-node communication is handled by a 100 Gbps InfiniBand network.

Baselines. Following prior work on optimizing distributed LLM training systems, we compare our system against two open-source state-of-the-art frameworks: DeepSpeed (Rasley et al., 2020) and Megatron-LM (Narayanan et al., 2021). Megatron-LM supports 4D parallelism, integrating tensor parallelism (TP, using Megatron-style sequence parallelism), pipeline parallelism (PP), data parallelism (DP, with ZeRO-1), and context parallelism (CP). DeepSpeed provides robust support for DeepSpeed ZeRO and Ulysses-style sequence parallelism. Both systems are primarily designed for homogeneous sequence lengths and are LLM training systems using a single static parallelism strategy while our method uses dynamic parallelism stategy.

Workloads. We evaluate our approach using the InternVL3 series (Chen et al., 2024) and Qwen3VL series models (Bai et al., 2025), ranging from 2B to 8B parameters, as the base models. For the training datasets, we employ three distinct ones: InternVid (Wang et al., 2024), OpenVid (Nan et al., 2025), and MSRVTT (Xu et al., 2016). Further details are provided in Appendix A.

Evaluation Protocol. For each baseline method, we tune the hybrid parallelism hyperparameters and select the best-performing configuration. For DHP, since it adaptively schedules parallelism parameters, no manual specification is required. To ensure a fair comparison, we fixed the global batch size to 512 in all experiments. The evaluation metrics are the end-to-end training iteration time and the token throughput per-device. During the performance test, we first warm up for 5 training steps, then record the average of the subsequent 10 steps for both metrics.

6.2. Evaluation

End-to-End Performance. To compare the end-to-end performance of DHP against the baselines, we conduct training on three multimodal datasets (MSRVTT, InternVid, and OpenVid) using InternVL3/2.5 and Qwen3VL models of varying sizes (2B, 4B, and 8B). As shown in Figure 6, DHP consistently outperforms the best baseline (DeepSpeed or Megatron-LM) across all configurations, achieving speedups ranging from $1.14\times$ (InternVL3-8B on MSRVTT) to $1.35\times$ (InternVL3-8B on OpenVid). The improvement is particularly pronounced on the diverse and complex OpenVid dataset, where DHP attains a $1.35\times$ speedup with the InternVL3-8B model and maintains robust gains (e.g., $1.28\times$ for InternVL2.5-4B and $1.32\times$ for Qwen3VL-8B). Notably, DHP delivers consistent speedups exceeding $1.2\times$ for 14 out of 18 configurations, with the highest accelerations observed on larger models (e.g., 8B) and more complex datasets like OpenVid. These results demonstrate DHP's effectiveness across diverse multimodal scenarios while scaling to larger model architectures.

Scalability Analysis. In this section, we analyze the performance of DHP on training clusters of different. We run experiments on 1, 2, 4, and 8 nodes (corresponding to 8, 16, 32, and 64 NPUs respectively) and record the token throughput. As shown in Figure 5 The results show that as the cluster scale increases, while per-device throughput decreases due to growing communication overhead, DHP's flexible and dynamic scheduling strategy mitigates this reduction effectively. Specifically, when scaling from 1 node (8 NPUs) to 8 nodes (64 NPUs), DHP maintains and even improves its relative throughput against DeepSpeed (1x) increasing from 1.02x to 1.16x. In contrast, DeepSpeed's absolute throughput drops from approximately 2.0 to 1.7 k tokens/s, and Megatron-LM's throughput declines from 1.8 to 1.52 k tokens/s. This demonstrates that DHP achieves superior scalability by minimizing the negative impact of communication overhead as the cluster size expands.

Generalization Performance Across Different Training Stages. To evaluate DHP's performance across different training stages, we freeze the vision encoder in MLLM and train the model on an 8-node cluster, measuring the end-to-end time. As illustrated in Figure 4, DHP delivers performance improvements across all evaluated datasets

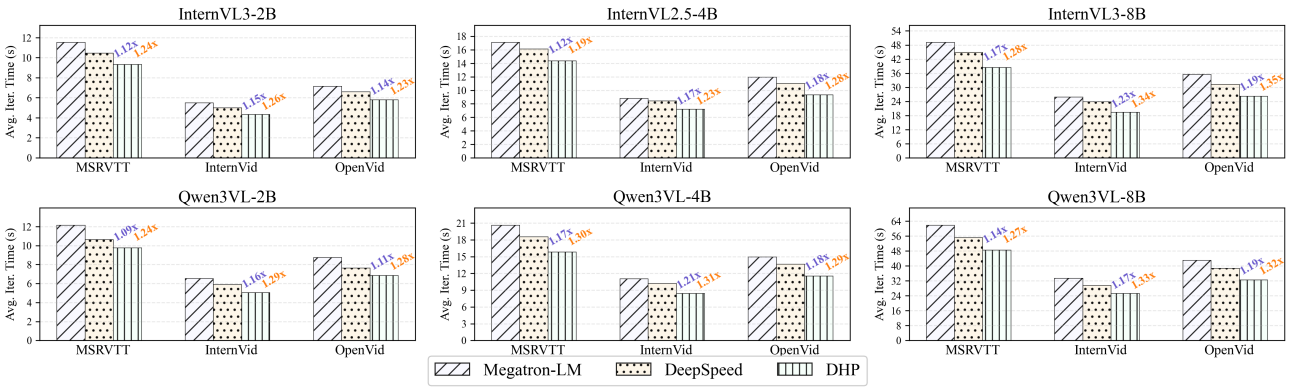

Figure 4. Average iteration time (in seconds) comparison across MSRVTT, InternVid, and OpenVid datasets for InternVL3-2B/8B, InternVL2.5-4B and Qwen3VL-2B/4B/8B models, evaluated with Megatron-LM (diagonal stripes), DeepSpeed (dots), and DHP (vertical stripes). Acceleration ratios (e.g., 1.29x) are annotated above bars, highlighting DHP and DeepSpeed's speedup over Megatron-LM.

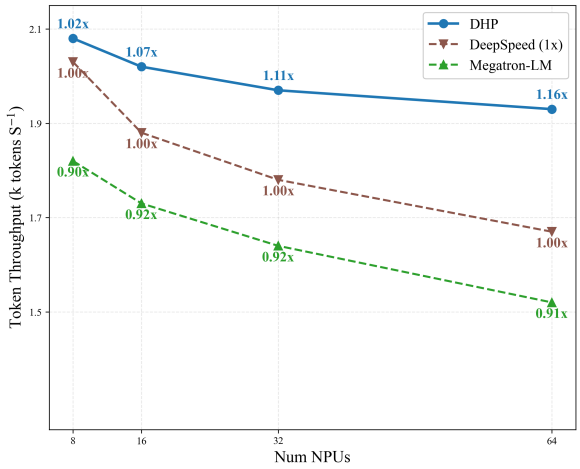

Figure 5. Token throughput (in k tokens/s) comparison across different NPU counts (8, 16, 32, and 64) for three training methods: DHP, DeepSpeed (1x), and Megatron-LM. DHP consistently achieves the highest throughput across all NPU configurations and exhibits a slight upward trend as NPU count increases, while DeepSpeed (1x) and Megatron-LM deliver lower throughput with relatively flat scaling behavior.

(MSRVTT, InternVid, and OpenVid) and model sizes (2B, 4B, and 8B), achieving speedups ranging from $1.11\times$ (Qwen3VL-2B on OpenVid) to $1.36\times$ (InternVL3-8B on OpenVid). The most significant gains are observed on larger models and more complex datasets: for instance, DHP accelerates training by $1.36\times$ for the InternVL3-8B model on OpenVid compared to Megatron-LM, and maintains robust speedups of $1.27\times$-$1.36\times$ across all 8B configurations. Notably, DHP deliver $1.36\times$ improvements on more complex OpenVid versus $1.28\times$ on MSRVTT. This consistent performance gain stems from DHP's comprehensive and precise cost modeling during scheduling, which dynamically adapts cost estimation strategies to different training stages (e.g., handling larger models with more complex communication patterns). The results demonstrate DHP's robust generalization across diverse training scenarios, maintaining efficiency even as model scale and dataset complexity increase.

Table 1. Time consumption under varying global batch sizes.

	Global Batch Size		
	128	256	512
Computing Time (s)	2.04	3.77	7.32
Schedule Time (ms)	468	684	921
Solver Time (ms)	21	44	86

Table 2. Time consumption of computation and scheduling under varying NPU numbers with GBS fixed at 512.

	NPU Number (GBS Fixed at 512)		
	16	32	64
Computing Time (s)	25.77	13.58	7.32
Schedule Time (ms)	294	549	921
Solver Time (ms)	20	43	86

6.3. Time Consumption of Solver

In the analysis of the method section, we emphasized the high efficiency of our algorithm with low time complexity. To prove the efficiency of our algorithm in real scenarios, we record empirical scheduling time, solver time consumption, and the shortest GBS computation time. As demonstrated in Tables 1 and 2, the solver time remains significantly low ($\leq$ 86 ms), confirming the scalability of the algorithm, and the scheduling time remains consistently shorter than the single GBS computation time in all scenarios—whether scaling the NPU count or increasing the GBS. For example, the scheduling time (468 ms) is less than 25% of the computation time (2.04 s) with GBS=128 and is only 0.9s when GBS=512. This negligible scheduling overhead (con-

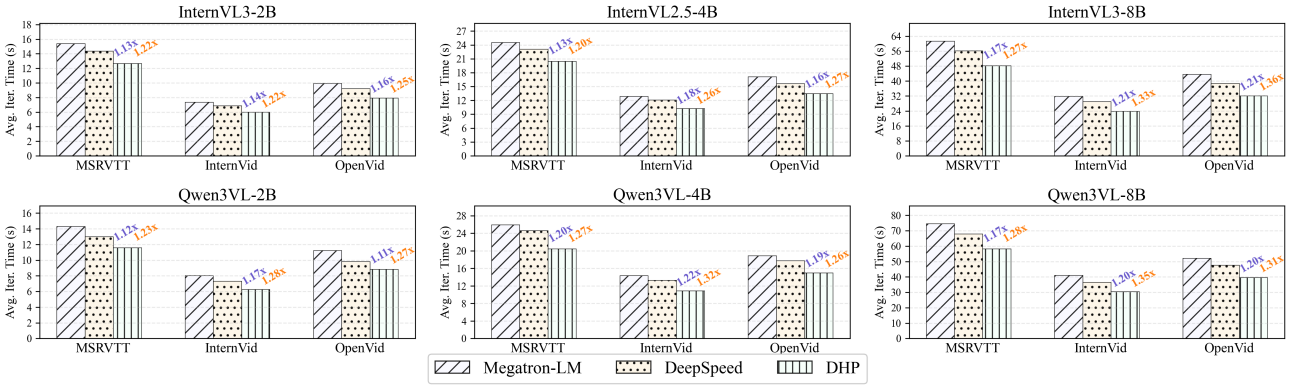

Figure 6. Average iteration time (in seconds) comparison for InternVL3-2B/8B, InternVL2.5-4B and Qwen3VL-2B/4B/8B models across MSRVTT, InternVid, and OpenVid datasets, with Megatron-LM, DeepSpeed, and DHP methods. Acceleration ratios (e.g., 1.22x) above bars indicate the performance improvement of DHP and DeepSpeed relative to Megatron-LM.

sistently below 1 second even at maximum scale) enables seamless overlap with computation, validating the practical efficiency of DHP.

6.4. Estimation Error of Cost Estimator

Table 3. Time cost estimation error (%) across different models and parameter scales

Model	Model Parameters		
	2B	4B	8B
Qwen3VL	7.93	6.71	4.27
InternVL3	7.48	6.54	4.12

To evaluate the accuracy of our Cost-Estimator and ensure reliable prediction information during scheduling, we conducted precision tests on different modules of the model and various computational processes, and ultimately computed the average error percentage. As shown in Table 3 we evaluate the accuracy of the cost estimator in DHP across different model scales, including 2B, 4B, and 8B parameter configurations. As can be seen, our overhead estimator adeptly approximates the execution overhead, with discrepancies consistently remaining below 8%. The precise estimations rendered by the estimator facilitate the performance optimization of DHP.

6.5. Case Study

To concretely illustrate DHP's dynamic partitioning of communication groups under varying data distributions, we present two actual parallel strategies employed within a single global batch size (GBS) across different training datasets. Case 1 in the table is derived from the OpenVid dataset, which exhibits a long-tailed and highly diverse data distribution, while Case 2 comes from MSRVTT, featuring a more uniform distribution yet still spanning a certain range of sequence lengths. As shown in Table 4, DeepSpeed and Megatron-LM statically partitions parallel groups based on the longest sequence length, whereas DHP adaptively selects appropriate parallel configurations tailored to the actual data length distribution. For more diverse data (Case 1), DHP employs a richer combination of context parallelism (CP) degrees, while for relatively uniform data (Case 2), the CP degrees remain more consistent. By mitigating the communication redundancy and load imbalance introduced by static parallel group partitioning, we achieve speedups of 1.17× (Case 1) and 1.14× (Case 2).

Table 4. Details of heterogeneous CP groups employed in each micro-batch of each case. Each $d \times m$ indicates we form m CP=d groups, and each $\langle \cdots \rangle \times x$ indicates the set of heterogeneous CP groups is employed for x micro-batches.

Case Study	Case 1	Case 2
DeepSpeed	$\langle 8 \rangle \times 4$	$\langle 4 \rangle \times 8$
Megatron-LM	$\langle 8 \rangle \times 4$	$\langle 4 \rangle \times 8$
DHP	$\langle 8 \rangle \times 1$ $\langle 6 \rangle \times 2$ $\langle 4 \rangle \times 1$ $\langle 2 \rangle \times 2$ $\langle 1 \rangle \times 4$	$\langle 4 \rangle \times 2$ $\langle 3 \rangle \times 4$ $\langle 2 \rangle \times 6$

7. Conclusion

In this paper, we presented Dynamic Hybrid Parallelism (DHP), an adaptive framework designed to optimize MLLM training on heterogeneous datasets. By enabling non-power-of-two parallelism degrees and employing a two-stage approximation algorithm, DHP effectively eliminates load imbalance and redundant communication with millisecond-level scheduling overhead. Our experimental results demonstrate that DHP achieves up to a 1.36× speedup in training throughput over Megatron-LM and DeepSpeed.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., et al. Gemini: A family of highly capable multimodal models, 2025. URL https://arxiv.org/abs/2312.11805.

Awadalla, A., Gao, I., Gardner, J., Hessel, J., Hanafy, Y., Zhu, W., Marathe, K., Bitton, Y., Gadre, S., Sagawa, S., Jitsev, J., Kornblith, S., Koh, P. W., Ilharco, G., Wortsman, M., and Schmidt, L. Openflamingo: An open-source framework for training large autoregressive vision-language models, 2023. URL https://arxiv.org/abs/2308.01390.

Bai, S., Cai, Y., Chen, R., Chen, K., et al. Qwen3-vl technical report, 2025. URL https://arxiv.org/abs/2511.21631.

Brandon, W., Nrusimha, A., Qian, K., Ankner, Z., Jin, T., Song, Z., and Ragan-Kelley, J. Striped attention: Faster ring attention for causal transformers, 2023. URL https://arxiv.org/abs/2311.09431.

Chen, Z., Wu, J., Wang, W., Su, W., Chen, G., Xing, S., Zhong, M., Zhang, Q., Zhu, X., Lu, L., Li, B., Luo, P., Lu, T., Qiao, Y., and Dai, J. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks, 2024. URL https://arxiv.org/abs/2312.14238.

Cheng, Z., Leng, S., Zhang, H., Xin, Y., Li, X., Chen, G., Zhu, Y., Zhang, W., Luo, Z., Zhao, D., and Bing, L. Videollama 2: Advancing spatial-temporal modeling and audio understanding in video-llms, 2024. URL https://arxiv.org/abs/2406.07476.

Dean, J., Corrado, G., Monga, R., Chen, K., Devin, M., Le, Q. V., Mao, M. Z., Ranzato, M., Senior, A. W., Tucker, P. A., Yang, K., and Ng, A. Y. Large scale distributed deep networks. In *In 26th Annual Conference on Neural Information Processing Systems 2012 (NeurIPS 2022)*, pp. 1232–1240, 2012.

Feng, W., Chen, Y., Wang, S., Peng, Y., Lin, H., and Yu, M. Optimus: Accelerating large-scale multi-modal llm training by bubble exploitation, 2025. URL https://arxiv.org/abs/2408.03505.

Ge, H., Feng, J., Huang, Q., Fu, F., Nie, X., Zuo, L., Lin, H., Cui, B., and Liu, X. Bytescale: Communication-efficient scaling of llm training with a 2048k context length on 16384 gpus. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pp. 963–978, New York, NY, USA, 2025. Association for Computing Machinery.

Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, M. X., Chen, D., Lee, H., Ngiam, J., Le, Q. V., Wu, Y., and Chen, Z. *GPipe: efficient training of giant neural networks using pipeline parallelism*. Curran Associates Inc., Red Hook, NY, USA, 2019.

Huang, Z., Ji, K., Gong, B., Qing, Z., Zhang, Q., Zheng, K., Wang, J., Chen, J., and Yang, M. Accelerating pre-training of multimodal llms via chain-of-sight. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2024.

Jacobs, S. A., Tanaka, M., Zhang, C., Zhang, M., Aminadabi, R. Y., Song, S. L., Rajbhandari, S., and He, Y. System optimizations for enabling training of extreme long sequence transformer models. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*, pp. 121–130, New York, NY, USA, 2024. Association for Computing Machinery.

Korthikanti, V., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models, 2022. URL https://arxiv.org/abs/2205.05198.

Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.

Li, B., Zhang, Y., Guo, D., Zhang, R., Li, F., Zhang, H., Zhang, K., Zhang, P., Li, Y., Liu, Z., and Li, C. LLaVA-onevision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.

Li, D., Shao, R., Xie, A., Xing, E., Gonzalez, J., Stoica, I., Ma, X., and Zhang, H. Lightseq: : Sequence level parallelism for distributed training of long context transformers. In *Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@NeurIPS 2023)*, 2023a.

Li, D., Shao, R., Xie, A., Xing, E. P., Ma, X., Stoica, I., Gonzalez, J. E., and Zhang, H. Distflashattn: Distributed memory-efficient attention for long-context llms training, 2024. URL https://arxiv.org/abs/2310.03294.

Li, S., Zhao, Y., Varma, R., Salpekar, O., Noordhuis, P., Li, T., Paszke, A., Smith, J., Vaughan, B., Damania, P., and Chintala, S. Pytorch distributed: experiences on

accelerating data parallel training. *Proc. VLDB Endow.*, 13(12):3005–3018, 2020.

Li, S., Xue, F., Baranwal, C., Li, Y., and You, Y. Sequence parallelism: Long sequence training from system perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2391–2404, Toronto, Canada, 2023b.

Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

Nan, K., Xie, R., Zhou, P., Fan, T., Yang, Z., Chen, Z., Li, X., Yang, J., and Tai, Y. Openvid-1m: A large-scale high-quality dataset for text-to-video generation. In *The Thirteenth International Conference on Learning Representations*, 2025.

Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N. R., Ganger, G. R., Gibbons, P. B., and Zaharia, M. Pipedream: generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pp. 1–15, New York, NY, USA, 2019. Association for Computing Machinery.

Narayanan, D., Phanishayee, A., Shi, K., Chen, X., and Zaharia, M. A. Memory-efficient pipeline-parallel dnn training. In *International Conference on Machine Learning*, 2020.

Narayanan, D., Shoeybi, M., Casper, J., et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2021.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. Learning transferable visual models from natural language supervision. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, pp. 8748–8763. PMLR, 2021.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020.

Rasley, J., Rajbhandari, S., Ruwase, O., and He, Y. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *SIGKDD*, pp. 3505–3506, 2020.

Shao, K., TAO, K., Qin, C., You, H., Sui, Y., and Wang, H. Holitom: Holistic token merging for fast video large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a.

Shao, K., Tao, K., Zhang, K., Feng, S., Cai, M., Shang, Y., You, H., Qin, C., Sui, Y., and Wang, H. When tokens talk too much: A survey of multimodal long-context token compression across images, videos, and audios, 2025b. URL https://arxiv.org/abs/2507.20198.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020. URL https://arxiv.org/abs/1909.08053.

Singh, A., Fry, A., Perelman, A., Tart, A., et al. Openai gpt-5 system card, 2025. URL https://arxiv.org/abs/2601.03267.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017.

Wang, W., Gao, Z., Gu, L., Pu, H., et al. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency, 2025a. URL https://arxiv.org/abs/2508.18265.

Wang, Y., He, Y., Li, Y., Li, K., Yu, J., Ma, X., Li, X., Chen, G., Chen, X., Wang, Y., He, C., Luo, P., Liu, Z., Wang, Y., Wang, L., and Qiao, Y. Internvid: A large-scale video-text dataset for multimodal understanding and generation, 2024. URL https://arxiv.org/abs/2307.06942.

Wang, Y., Wang, S., Zhu, S., Fu, F., Liu, X., Xiao, X., Li, H., Li, J., Wu, F., and Cui, B. Flexsp: Accelerating large language model training via flexible sequence parallelism. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 421–436, New York, NY, USA, 2025b. Association for Computing Machinery.

Xing, L., Huang, Q., Dong, X., Lu, J., Zhang, P., Zang, Y., Cao, Y., He, C., Wang, J., Wu, F., and Lin, D. Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction, 2025. URL https://arxiv.org/abs/2410.17247.

Xu, J., Mei, T., Yao, T., and Rui, Y. Msr-vtt: A large video description dataset for bridging video and language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

Xu, J., Guo, Z., He, J., Hu, H., He, T., Bai, S., Chen, K., Wang, J., Fan, Y., Dang, K., Zhang, B., Wang, X., Chu, Y., and Lin, J. Qwen2.5-omni technical report, 2025. URL https://arxiv.org/abs/2503.20215.

Yang, S., Chen, Y., Tian, Z., Wang, C., Li, J., Yu, B., and Jia, J. Visionzip: Longer is better but not necessary in vision language models. *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 19792–19802, 2024.

Zhang, B., Li, K., Cheng, Z., Hu, Z., Yuan, Y., Chen, G., Leng, S., Jiang, Y., Zhang, H., Li, X., Jin, P., Zhang, W., Wang, F., Bing, L., and Zhao, D. Videollama 3: Frontier multimodal foundation models for image and video understanding, 2025. URL https://arxiv.org/abs/2501.13106.

Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., and Li, S. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *Proc. VLDB Endow.*, 16:3848–3860, 2023. URL https://doi.org/10.14778/3611540.3611569.

A. Details of Experimental Setups

A.1. Model Configuration

Table 5. **Models for evaluation.**

Model	#Layers	#Heads	#Groups	Hidden Dim	Vision Hidden Dim
InternVL3-2B	28	12	2	1536	1024
InternVL2.5-4B	36	16	8	2048	1024
InternVL3-8B	28	28	4	3584	1024
Qwen3-VL-2B	28	16	8	2048	1024
Qwen3-VL-4B	36	32	8	2560	1024
Qwen3-VL-8B	36	32	8	4096	1152

A.2. Details of Baselines

- **Deepspeed-Ulysses** (Jacobs et al., 2024) addresses the memory constraints imposed by extreme sequence lengths in LLM training through a novel sequence parallelism mechanism involving hybrid partitioning. In contrast to Megatron-SP (Korthikanti et al., 2022), which partitions the sequence dimension only for LayerNorm and Dropout while relying on ring-based communication for Self-Attention, DeepSpeed Ulysses partitions the input data along the sequence dimension across all modules. To enable global attention without storing the full sequence on a single device, it employs an all-to-all collective communication primitive to transform the data layout.

- **Megatron-LM** (Shoeybi et al., 2020) composes tensor, pipeline, and data parallelism (PTD-P) to scale language model training to trillions of parameters. To address the significant idle time (bubble) inherent in standard pipeline parallelism, it proposes a novel interleaved pipelining schedule. Instead of assigning a contiguous set of layers to each device, this schedule assigns v virtual model chunks to each device. This allows devices to perform computation on multiple stages simultaneously, reducing the pipeline bubble size by a factor of v. Formally, with p pipeline stages and m micro-batches, the bubble fraction is reduced to:

$$\text{Bubble Fraction} \approx \frac{1}{v}\left(\frac{p-1}{m}\right).$$

  This optimization ensures that the model parameters θ are updated with significantly higher NPU throughput compared to non-interleaved schedules.

A.3. Details of Datasets

We detail the specifications of the three datasets, including InternVid, OpenVid, and MSRVTT.

- **InternVid** comprises 10 million video clips paired with generated high-quality captions sourced from publicly available web videos, facilitating large-scale video-language pre-training and representation learning. The dataset is available at `https://huggingface.co/datasets/OpenGVLab/InternVid`.

- **OpenVid** comprises a curated collection of high-aesthetic video clips with a minimum resolution of 512×512, tailored for research institutions to enhance visual clarity, facilitating high-fidelity text-to-video training and quality-tuning as a complement to existing datasets. The dataset is available at `https://huggingface.co/datasets/nkp37/OpenVid-1M`.

- **MSRVTT** consists of 10,000 video clips paired with 200,000 natural language descriptions from 20 diverse categories, measuring the video-to-text generation and multimodal understanding capabilities of machine learning models. The dataset is available at `https://huggingface.co/datasets/friedrichor/MSR-VTT`.